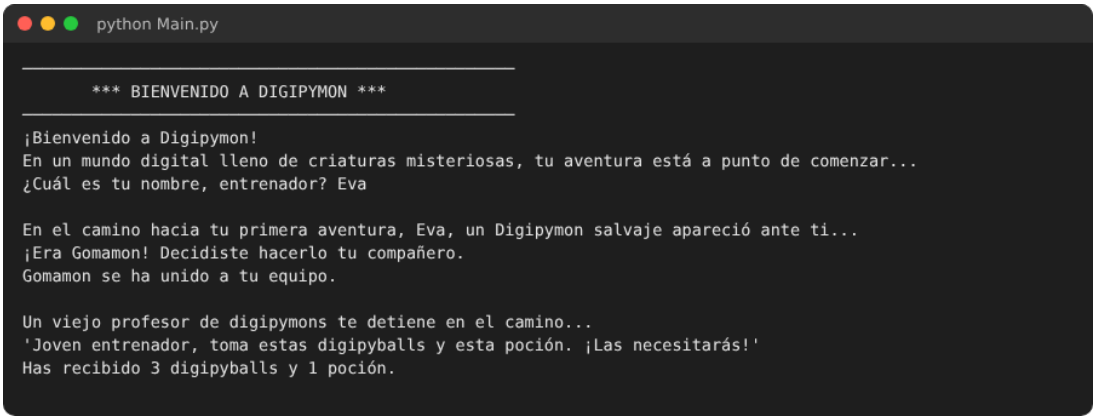


Digipymon

RPG por consola en Python

Eva Gallardo Romero

24 de mayo de 2026

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Inicio . . . 3
3.2 Buscar Digipymon . . . 3
3.3 Combate . . . 3
3.4 Digishop . . . 4
3.5 Usar objetos, inventario y equipo . . . 4

4 Requisitos del sistema **4**
4.1 Requisitos funcionales . . . 4
4.2 Requisitos no funcionales . . . 5

5 Documentación técnica **5**
5.1 Digipymon.py . . . 5
5.2 Jugador.py . . . 6
5.3 Inventario.py . . . 7
5.4 Enemigo.py y ListaNombres.py . . . 7
5.5 Main.py . . . 7

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Digipymon.

La documentación del código generada con `pydoc` está en evagr11.github.io/Digipymon.

1 Descripción general

Digipymon es un **videojuego de rol por consola** hecho en **Python 3** e inspirado en Digimon. El jugador empieza con un compañero aleatorio, tres *digipyballs* y una poción, y desde un menú puede:

1. **Buscar Digipymon** salvajes e intentar capturarlos (hasta seis en el equipo).
2. **Luchar contra un entrenador**, que saca tantos Digipymon como tenga el jugador.
3. Comprar en la **Digishop** con *digicoins*: digipyballs, pociones, anabolizantes y objetos para subir de nivel.
4. **Usar objetos** en sus Digipymon.
5. Consultar el **inventario** y el **equipo**.

Fue la **práctica final de Programación** de 1.º de DAM en New Digital Talent (Granada). El enunciado daba las clases (`Digipymon`, `Jugador`, `Enemigo`, `Inventario` y `ListaNombres`) y el funcionamiento básico del menú. Sobre esa base añadí la narrativa con el texto que aparece letra a letra y tres mejoras propias:

- **Bonus de nivel**: el ataque efectivo suma `nivel × 0.10` al ataque base.
- **Evolución**: al llegar a nivel 5, el Digipymon pasa a llamarse «Mega…» y gana 15 de vida y 5 de ataque.
- **Ventaja de tipos**: fuego gana a planta, planta a agua y agua a fuego; la ventaja suma un 30 % del ataque base.

2 Diagrama de clases

`Main.py` contiene el bucle del juego y una función por cada opción del menú. Crea el `Jugador` y su `Inventario` al empezar, y un `Enemigo` nuevo en cada combate. Tanto el jugador como el enemigo guardan una lista de objetos `Digipymon`, y los nombres aleatorios salen de `ListaNombres`.

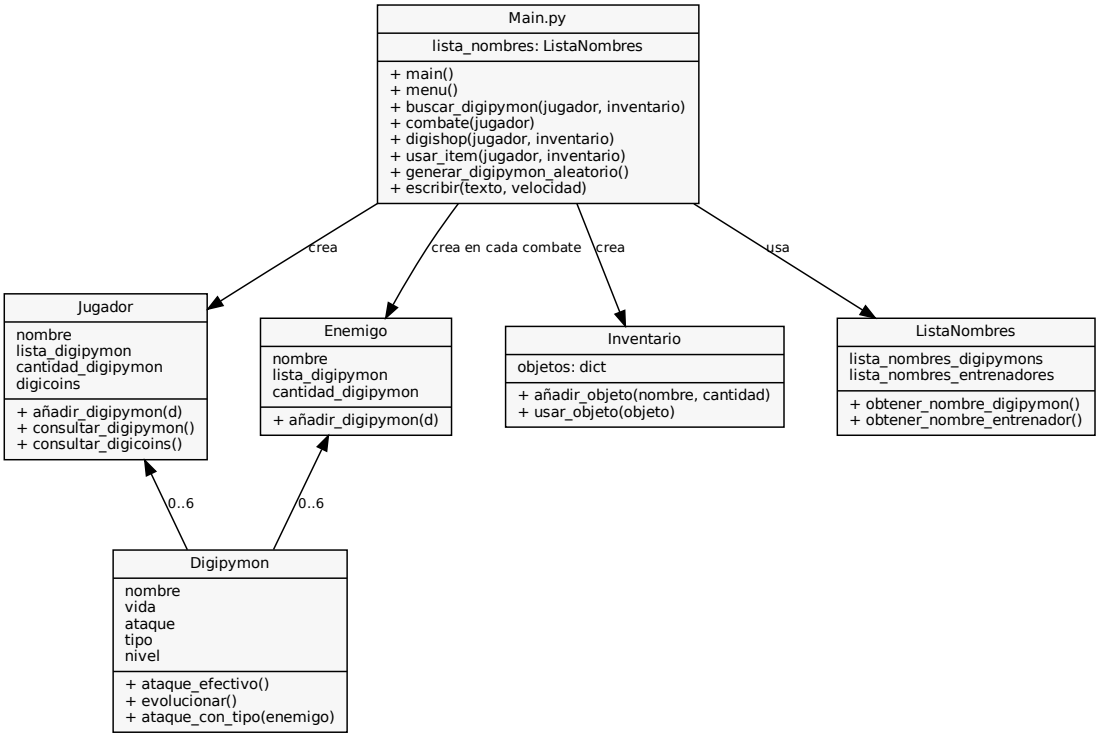


Figura 1: Diagrama de clases de Digipymon

3 Diseño

Al ser un juego de consola, el diseño está en el texto: líneas separadoras, títulos en mayúsculas entre asteriscos y mensajes que se escriben **letra a letra** con la función `escribir`, como en los juegos clásicos. Las capturas son de una partida real.

3.1 Inicio

La bienvenida cuenta la historia, pide el nombre del entrenador y regala el primer Digipymon, tres digipyballs y una poción. Después aparece el **menú principal** con siete opciones.

3.2 Buscar Digipymon

Aparece un Digipymon salvaje con todos sus datos y la probabilidad de captura (`100 - nivel × 10`). Si quedan digipyballs y hay hueco en el equipo, se puede lanzar una; la digipyball se gasta aunque falle.

3.3 Combate

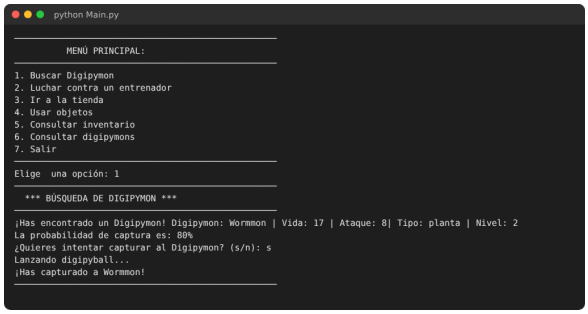
Cada Digipymon del jugador se enfrenta al del rival que está en la misma posición. Para cada combate se muestran las estadísticas base, el bonus de nivel, la ventaja de tipo si la hay y el resumen final. Gana el que tenga más ataque total.

3.4 Digishop

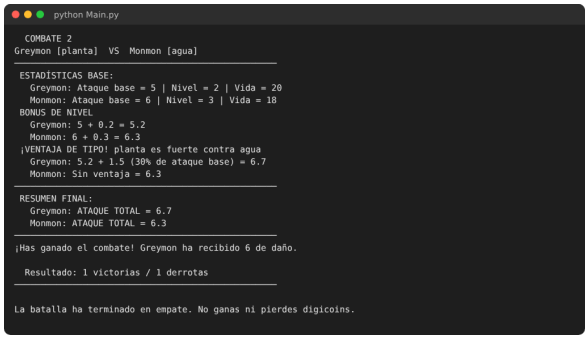
Lista los cuatro objetos con su precio y deja comprar varios seguidos hasta elegir *Salir de la tienda*.

3.5 Usar objetos, inventario y equipo

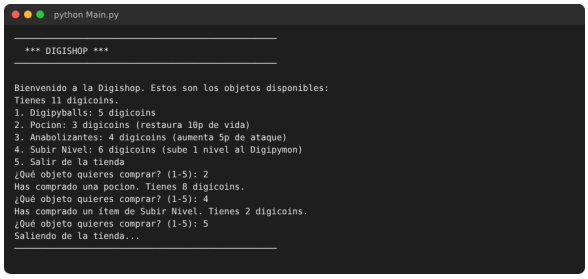
Se elige un objeto y el Digipymon en el que se usa. Las digipyballs no se pueden usar aquí. El inventario y el equipo se consultan desde el menú.



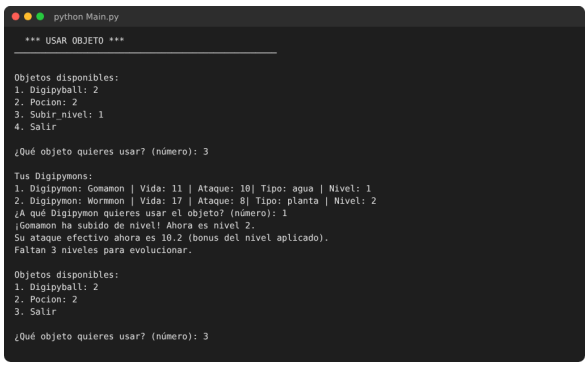
3.1. Menú y búsqueda



3.3. Combate con ventaja de tipo



3.4. Digishop



3.5. Usar un objeto



3.5. Inventario y equipo

Figura 2: Capturas de una partida

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	El juego debe pedir el **nombre del entrenador** y darle un Digipymon aleatorio, 3 digipyballs y 1 poción.
RF02	Los Digipymon se deben generar con **vida** de 10 a 20, **ataque** de 1 a 10, **nivel** de 1 a 3, tipo y nombre aleatorios.

ID	Requisito funcional
RF03	Al buscar, la probabilidad de captura debe ser **100 - nivel × 10** y cada intento debe gastar una digipyball.
RF04	El jugador no puede tener más de **6 Digipymon**.
RF05	El entrenador rival debe tener **tantos Digipymon** como el jugador.
RF06	En el combate se debe aplicar el **bonus de nivel** y la **ventaja de tipo** (+30 % del ataque base).
RF07	Ganar una batalla debe dar un digicoin por victoria y perderla quitar uno por derrota; huir cuesta 1 digicoin. Los digicoins nunca bajan de 0.
RF08	La Digishop debe vender digipyballs (5), pociones (3), anabolizantes (4) y subir nivel (6), comprobando que hay digicoins suficientes.
RF09	La poción debe sumar **10 de vida**, los anabolizantes **5 de ataque** y subir nivel **1 nivel**.
RF10	Al llegar a **nivel 5**, el Digipymon debe **evolucionar**.
RF11	Un objeto debe desaparecer del inventario cuando su cantidad llegue a 0.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El juego debe funcionar en la **consola** con **Python 3** y sin librerías externas.
RNF02	El código debe estar organizado en **clases**, una por archivo.
RNF03	Los textos deben aparecer **letra a letra** para dar ritmo a la partida.
RNF04	Importar `Main.py` no debe arrancar el juego (bloque `if __name__ == "__main__"`), para poder generar la documentación con `pydoc`.
RNF05	Todas las clases y funciones deben tener **docstrings**.

5 Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `import` y los docstrings, que repiten la descripción.

5.1 Digipymon.py

5.1.1 Ubicación

`Digipymon/Digipymon.py`

Descripción general:
La clase `Digipymon` representa una criatura con nombre, vida, ataque, tipo y nivel. Contiene las tres mejoras propias del juego.

5.1.2 Funciones

ataque_efectivo **Descripción:** Devuelve el ataque base más el bonus de nivel.
Código comentado:

```
def ataque_efectivo(self):
    bonus = self.nivel * 0.10          # 0.1 puntos por cada nivel
    return self.ataque + bonus
```

evolucionar **Descripción:** Añade «Mega» al nombre y sube la vida y el ataque. Devuelve el nombre anterior para poder mostrar el mensaje.

Código comentado:

```
def evolucionar(self):
    nombre_original = self.nombre
    self.nombre = "Mega" + self.nombre
    self.vida += 15
    self.ataque += 5
    return nombre_original
```

ataque_con_tipo **Descripción:** Calcula el ataque contra otro Digipymon. Si su tipo gana al del rival, suma un 30 % del ataque base. Devuelve el ataque, si hubo ventaja y el bonus.

Código comentado:

```
def ataque_con_tipo(self, enemigo_digipymon):
    ataque_con_nivel = self.ataque_efectivo()
    bonus_tipo = self.ataque * 0.30        # +30% sobre el ataque BASE

    # Cada tipo gana al que tiene como valor
    ventajas = {
        "fuego": "planta",
        "planta": "agua",
        "agua": "fuego"
    }

    if ventajas.get(self.tipo) == enemigo_digipymon.tipo:
        return ataque_con_nivel + bonus_tipo, "ventaja", bonus_tipo
    else:
        return ataque_con_nivel, "normal", 0
```

5.2 Jugador.py

5.2.1 Ubicación

`Digipymon/Jugador.py`

Descripción general:
La clase `Jugador` guarda el nombre, la lista de Digipymon, cuántos tiene y sus digicoins, que empiezan en 10.

5.2.2 Funciones

añadir_digipymon **Descripción:** Añade un Digipymon al equipo y actualiza el contador.

Código comentado:

```
def añadir_digipymon(self, digipymon):
    self.lista_digipymon.append(digipymon)
    self.cantidad_digipymon += 1
```

consultar_digipymon **Descripción:** Imprime el equipo o avisa si está vacío.

Código comentado:

```
def consultar_digipymon(self):
    if not self.lista_digipymon:
        print(f"No tienes ningún Digipymon {self.nombre}.")
    else:
        print(f"  Digipymons de {self.nombre}:")
        for digipymon in self.lista_digipymon:
            print(digipymon)            # usa __str__ de Digipymon
```

5.3 Inventario.py

5.3.1 Ubicación

`Digipymon/Inventario.py`

Descripción general:
La clase `Inventario` guarda los objetos en un diccionario `{nombre: cantidad}`.

5.3.2 Funciones

añadir_objeto **Descripción:** Suma la cantidad si el objeto ya estaba o crea la entrada si no.

Código comentado:

```
def añadir_objeto(self, nombre, cantidad):
    if nombre in self.objetos:
        self.objetos[nombre] += cantidad
    else:
        self.objetos[nombre] = cantidad
```

usar_objeto **Descripción:** Gasta una unidad y quita el objeto cuando se acaba.

Código comentado:

```
def usar_objeto(self, objeto):
    if objeto in self.objetos:
        self.objetos[objeto] -= 1
        if self.objetos[objeto] == 0:
            self.objetos.pop(objeto)    # no se deja a 0 en el inventario
```

5.4 Enemigo.py y ListaNombres.py

5.4.1 Ubicación

`Digipymon/Enemigo.py, Digipymon/ListaNombres.py`

Descripción general:
`Enemigo` es el entrenador rival: un nombre y su lista de Digipymon, con el mismo `añadir_digipymon` que el jugador. `ListaNombres` tiene veinte nombres de Digipymon y veinte de entrenadores, y devuelve uno al azar.

5.4.2 Funciones

obtener_nombre_digipymon **Descripción:** Elige un nombre al azar de la lista.

Código comentado:

```
def obtener_nombre_digipymon(self):
    return random.choice(self.lista_nombres_digipymons)
```

5.5 Main.py

5.5.1 Ubicación

`Digipymon/Main.py`

Descripción general:
Es el programa principal: la narrativa, el menú y una función por cada opción.

5.5.2 Funciones

escribir **Descripción:** Imprime un texto letra a letra con una pequeña pausa.
Código comentado:

```
def escribir(texto, velocidad=0.03):
    for letra in texto:
        print(letra, end='', flush=True)    # flush para que salga al momento
        time.sleep(velocidad)
    print()
```

generar_digipymon_aleatorio **Descripción:** Crea un Digipymon con valores al azar.
Código comentado:

```
def generar_digipymon_aleatorio():
    vida = random.randint(10, 20)
    ataque = random.randint(1, 10)
    nivel = random.randint(1, 3)
    tipo = random.choice(["fuego", "agua", "planta"])
    nombre = lista_nombres.obtener_nombre_digipymon()
    return Digipymon(nombre, vida, ataque, tipo, nivel)
```

buscar_digipymon **Descripción:** Genera un Digipymon salvaje y deja intentar capturarlo si hay digipyballs y sitio en el equipo.
Código comentado:

```
def buscar_digipymon(jugador, inventario):
    digipymon = generar_digipymon_aleatorio()
    escribir(f"¡Has encontrado un Digipymon! {digipymon}")
    probabilidad_captura = 100 - (digipymon.nivel * 10)
    escribir(f"La probabilidad de captura es: {probabilidad_captura}%")

    cantidad_digipyballs = 0
    if "digipyball" in inventario.objetos:
        cantidad_digipyballs = inventario.objetos["digipyball"]
    if cantidad_digipyballs <= 0:
        escribir("No tienes digipyballs para capturar al Digipymon.")
        return
    if jugador.cantidad_digipymon >= 6:
        escribir("No puedes capturar más Digipymons. Tu equipo está lleno.")
        return

    opcion = input("¿Quieres intentar capturar al Digipymon? (s/n): ").lower()
    if opcion == 's':
        inventario.usar_objeto("digipyball")          # se gasta aunque falle
        if random.randint(1, 100) <= probabilidad_captura:
            jugador.añadir_digipymon(digipymon)
            escribir(f"¡Has capturado a {digipymon.nombre}!")
        else:
            escribir(f"¡Fallaste al capturar a {digipymon.nombre}!")
    else:
        escribir("Has huido del Digipymon.")
```

combate **Descripción:** Crea un entrenador con tantos Digipymon como el jugador y los enfrenta por orden. Se omiten los mensajes que explican cada combate en pantalla.
Código comentado:

```
def combate(jugador):
    enemigo = Enemigo(lista_nombres.obtener_nombre_entrenador())
    for digipymon in jugador.lista_digipymon:            # mismo número que el jugador
        enemigo.añadir_digipymon(generar_digipymon_aleatorio())

    opcion = input("¿Quieres combatir contra el entrenador? (s/n): ").lower()
    if opcion != 's':
        jugador.digicoins = jugador.digicoins - 1        # huir cuesta 1 digicoin
        if jugador.digicoins < 0:
            jugador.digicoins = 0
        return

```

```
    victorias = 0
    derrotas = 0
    cantidad_combates = jugador.cantidad_digipymon
    for i in range(cantidad_combates):
        jugador_digipymon = jugador.lista_digipymon[i]
        enemigo_digipymon = enemigo.lista_digipymon[i]
        ataque_jugador = jugador_digipymon.ataque_con_tipo(enemigo_digipymon)[0]
        ataque_enemigo = enemigo_digipymon.ataque_con_tipo(jugador_digipymon)[0]

        if jugador_digipymon.vida <= 0:                  # debilitado: pierde
            derrotas += 1
        elif ataque_jugador > ataque_enemigo:
            victorias += 1
            daño_recibido = int(ataque_enemigo)
            jugador_digipymon.vida = jugador_digipymon.vida - daño_recibido
        elif ataque_jugador < ataque_enemigo:
            derrotas += 1
            daño_recibido = int(ataque_enemigo - ataque_jugador)
            jugador_digipymon.vida = jugador_digipymon.vida - daño_recibido
        # (la vida no baja de 0; si empatan, nadie recibe daño)

    if victorias > derrotas:
        jugador.digicoins += victorias
    elif derrotas > victorias:
        jugador.digicoins = jugador.digicoins - derrotas
        if jugador.digicoins < 0:
            jugador.digicoins = 0
```

usar_item (subir nivel) **Descripción:** Fragmento que sube el nivel y lanza la evolución al llegar a 5.

Código comentado:

```
elif item_elegido == "subir_nivel":
    digipymon.nivel += 1
    if digipymon.nivel >= 5:
        nombre_anterior = digipymon.evolucionar()
        escribir(f"¡{nombre_anterior} ha evolucionado a {digipymon.nombre}!")
    else:
        niveles_restantes = 5 - digipymon.nivel
        escribir(f"Faltan {niveles_restantes} niveles para evolucionar.")
inventario.usar_objeto(item_elegido)
```

main **Descripción:** Cuenta la introducción, prepara al jugador y repite el menú hasta elegir Salir.

Código comentado:

```
def main():
    nombre_jugador = input("¿Cuál es tu nombre, entrenador? ")
    jugador = Jugador(nombre_jugador)
    inventario = Inventario()

    jugador.añadir_digipymon(generar_digipymon_aleatorio())   # compañero inicial
    inventario.añadir_objeto("digipyball", 3)
    inventario.añadir_objeto("pocion", 1)

    while True:
        opcion = menu()
        if opcion == '1':
            buscar_digipymon(jugador, inventario)
        elif opcion == '2':
            combate(jugador)
        elif opcion == '3':
            digishop(jugador, inventario)
        elif opcion == '4':
            usar_item(jugador, inventario)
        # 5 inventario, 6 equipo...
        elif opcion == '7':
            break

if __name__ == "__main__":    # al importarlo (pydoc) no arranca el juego
    main()
```